

JUSTIFICATION FOR ARCHITECTURAL PATTERN SELECTION

Hyperlocal Courier Dispatch & Tracking Engine

Name : Dharoor Shiva Sai Varun

SRN : PES1UG24CS150

1. Architectural Styles Evaluated

Style	Advantages	Limitations for This System	Suitability
Monolithic	Simple development and deployment; low initial complexity.	Tightly coupled functions make independent scaling, deployment, and fault isolation difficult.	Low
Layered	Clear separation of presentation, business logic, and data access.	Good for internal organization, but weaker independent deployment and service-level fault isolation.	Medium
Microservices	Independent deployment, targeted scaling, fault isolation, clear service boundaries, flexible integrations.	Higher distributed-system and operational complexity.	High
Event-Driven	Strong asynchronous communication and loose coupling for notifications and delivery events.	Does not by itself organize all synchronous request, matching, tracking, and verification responsibilities.	Medium-High

2. Selected Architecture: Microservices

Microservices Architecture is selected as the most appropriate style because the courier platform consists of separate business capabilities with different responsibilities and workload characteristics. The proposed design decomposes the system into Delivery Request Service, Rider Matching & Dispatch, Real-Time Tracking, OTP & Delivery Verification, Route Optimization, and Notification Service. Each service exposes a focused responsibility through well-defined interfaces, reducing unnecessary coupling between modules.

3. Reasons for Selecting Microservices

First, the architecture supports independent scalability. Real-Time Tracking receives frequent GPS and status updates and has a target latency below 2 seconds, whereas Rider Matching performs location-based assignment using the nearest-rider radius of 3 km. These services can be scaled independently according to their actual workload instead of scaling the entire application.

Second, microservices improve fault isolation and maintainability. A failure in the Notification Service or external Mapping/Routing API can be isolated without bringing down the complete delivery workflow. Individual services can also be updated or deployed without requiring every other module to be redeployed, provided their interfaces remain compatible. This is particularly useful for a courier platform where tracking, dispatch, and verification have different operational requirements.

4. Component Interaction and Data Flow

The Sender App and Rider App communicate through the API Gateway, which handles authentication, routing, and rate limiting. A delivery request is forwarded to the Delivery Request Service and then to Rider Matching & Dispatch for assignment or reassignment. The Rider App continuously sends GPS and status telemetry to Real-Time Tracking, which provides live tracking updates and stores GPS history in the Telemetry Store. Route Optimization sends routing requests to the external Mapping/Routing API. At delivery completion, OTP & Delivery Verification validates the OTP and records proof-of-delivery metadata. The Notification Service then sends status or confirmation messages through the external SMS/Push Provider.

5. Security, Performance and Quality Attributes

The design supports security, performance, availability, and maintainability. The API Gateway provides a controlled entry point for authentication and request routing. OTP and GPS information requires TLS 1.2+ while in transit and AES-256 while at rest. Separating the Delivery Database from the Telemetry Store allows operational delivery data and high-frequency tracking data to be managed independently. The architecture therefore supports targeted scaling, clear ownership of responsibilities, and improved recovery from individual service failures. Payment processing and rider onboarding/KYC are outside the defined system scope.

6. Conclusion

Overall, Microservices Architecture provides the best fit for the Hyperlocal Courier Dispatch & Tracking Engine. It balances clear component responsibilities with independent scalability, fault isolation, real-time performance, security, and integration flexibility. The UML component diagram reflects these decisions through explicit service interfaces, data stores, and external service dependencies.